

DATA 298A — MSDA Project I

PROJECT REPORT

Team 7

Aayushi Parag Shah
Amaan Arif Khan
Dhruva Jigneshkumar Joshi
Rohit Raju Dhole
Sneha Guravannavar

December 8 2025

1.1 Project Background and Executive Summary

The role of Large Language Models (LLMs) has been expanding within the area of healthcare, drug safety monitoring (pharmacovigilance) and biomedical information systems. These large models have the potential to both help and harm workflows based on their ability to generate what are considered “facts” when working with structured sources like DrugBank, making random changes to the schema and producing results that could put patients at risk due to inconsistency. This will be particularly problematic because biomedical datasets require a very high degree of factual accuracy, schema consistency, and controlled readability.

DrugBank has a large hierarchical organization and contains a great deal of complex information about how the thousands of drugs actually work at the chemical, molecular, and clinical levels. While traditional ETL methods allow us to parse and standardise this complex information, they struggle to automatically add high-value clinical context (mechanisms of action, contraindications, warnings, summaries) without extensive human annotation.

Although LLMs can enrich structured datasets with contextual explanations, summaries, and harmonized fields, their limitations—hallucinations, inconsistent JSON output, schema drift—pose risks for real biomedical workflows. This motivates the central research question:

“How can we reliably use LLMs to enrich biomedical datasets without hallucinations, schema breakages, or unsafe drug information generation?”

Project Objective

This project develops a **production-grade LLM Guardrails Pipeline** that:

- Ingests DrugBank XML into a **MinIO-backed Bronze layer**
- Parses XML into **structured Silver Delta tables** using PySpark
- Performs controlled LLM enrichment with **multiple models (OpenAI, Gemini, Groq)**
- Selects **Groq Mixtral 8×7B** and **LLaMA-3.1 70B** for production
- Enforces **robust JSON guardrails**, schema validation, completeness checks, hallucination detection
- Stores validated enriched outputs in **Delta Lake Gold tables**
- Prepares **train/val/test datasets** for future ML tasks
- Provides EDA, statistics, and reproducible Airflow orchestration

High-Level Approach

The system is implemented as an **eight-stage workflow**:

1. **Ingestion Layer** — DrugBank XML and logs stored in MinIO Bronze
2. **Parsing & Transformation** — XML → Silver Delta with PySpark
3. **LLM Enrichment** — multi-model comparison → Groq-based JSON mode
4. **Guardrails Enforcement** — schema checks, hallucination filters, retries
5. **Delta Lake Governance** — Bronze/Silver/Gold ACID framework
6. **Airflow Orchestration** — DAGs for automated ETL and enrichment
7. **Data Quality Checks** — quality thresholds for Silver → Gold promotion
8. **ML Preparation** — tokenization, augmentation, dataset splitting

Expected Contributions and Applications

- A **reusable LLM guardrails architecture** for structured JSON generation in biomedical domains
- A scalable **DrugBank parsing + enrichment ETL system**
- A **model comparison study** on hallucination rates, JSON validity, throughput, cost
- A curated **Gold dataset** ready for analytics and ML
- A complete **feature engineering and text-tokenization pipeline** for future classification/QA models

1.2 Project Requirements

1.2.1 Functional Requirements

ID	Requirement
FR1	Ingest DrugBank XML into object storage (MinIO, Bronze layer).

FR2	Convert XML to structured parquet/Delta in Silver layer.
FR3	Enrich drug records using LLM models.
FR4	Validate enriched JSON using a predefined schema.
FR5	Store enriched outputs as JSONL in MinIO (Silver).
FR6	Promote Silver → Gold based on DQ rules and thresholds.
FR7	Develop model evaluation metrics for LLM performance.
FR8	Generate EDA, descriptive statistics, visualizations.
FR9	Automate the entire pipeline using Airflow.

1.2.2 Non-Functional Requirements

ID	Requirement
NFR1	Deterministic, schema-consistent LLM output.
NFR2	Low-latency, high-throughput inference (Groq LPUs).

NFR3	Cost-efficient execution (prefer free/open models).
NFR4	ACID-compliant Delta Lake storage.
NFR5	Fault tolerance via retry/fallback logic.
NFR6	Scalable deployment using Docker + MinIO.
NFR7	Explainable outputs through logs and validation reports.

1.3 Project Deliverables

- **D1:** Full LLM enrichment pipeline (code + config)
- **D2:** Silver & Gold DrugBank Delta tables
- **D3:** Guardrails scripts — JSON schema checks, hallucination filters
- **D4:** Airflow DAGs: ingestion, transformation, enrichment, promotion
- **D5:** EDA visualizations (PNG)
- **D6:** Feature engineering + tokenization pipeline
- **D7:** Train/validation/test dataset slices

1.4 Technology and Solution Survey

1.4.1 Core Technologies

- **MinIO** — object storage
- **PySpark 3.5.3** — distributed ETL
- **Delta Lake 3.2.1** — ACID bronze/silver/gold architecture
- **Apache Kafka** — future streaming integration
- **Apache Airflow 2.9.0** — end-to-end orchestration
- **Groq LLMs: Mixtral 8×7B + LLaMA-3.1 70B** — production models
- **OpenAI GPT-4o, GPT-4.1 Mini, Gemini Flash/Pro** — baseline comparison
- **Streamlit** — interactive analytics UI

1.4.2 LLM Solution Comparison

Model	Strengths	Weaknesses	Outcome
GPT-4o	Excellent language understanding	Expensive; unstable JSON	Rejected
GPT-4.1 Mini	Cheap & fast	Schema breakages	Rejected
Gemini 1.5 Flash	Fast; long-context	Inconsistent medical accuracy	Rejected
Gemini 1.5 Pro	Better reasoning	Costly; JSON instability	Rejected
Groq Mixtral	Very fast; deterministic JSON	Slightly less fluent	Accepted
Groq LLaMA-3.1 70B	Highest biomedical accuracy	Higher latency	Accepted

1.5 Literature Survey of Existing Research

Paper / Source	Summary	Relevance
Vaswani et al., <i>Attention Is All You Need</i>	Introduced Transformer	Foundation for modern LLMs
OpenAI GPT-4 Report	Describes GPT-4 capabilities & hallucination risks	Motivates guardrails
Google Gemini Whitepaper	Long-context models, JSON-mode limits	Informs baseline evaluation
Meta LLaMA-3 Model Card	Stability & biomedical reasoning	Supports model selection
Delta Lake Whitepaper	ACID transactions for lakehouse	Justifies bronze/silver/gold
Lin et al., Hallucination Survey	Techniques for reducing hallucinations	Guides guardrail design
DrugBank Documentation	Schema & semantics of DrugBank	Defines the raw data source

A complete IEEE reference list is at the end of the report.

2.1 Data Management Plan

This project employs a **data lakehouse architecture** using MinIO object storage and Delta Lake ACID tables. DrugBank XML and supporting parquet datasets are incrementally ingested into the Bronze layer, then transformed, enriched, validated, and promoted to Silver and Gold layers.

2.1.1 Data Collection Approaches

Sources of Data

Data Source	Format	Description
DrugBank Raw Dataset	XML	Primary hierarchical biomedical dataset containing detailed drug metadata.
Chat Query Logs	Parquet	Synthetic user queries for drug-information search simulation.
PostgreSQL Orders	Parquet	Simulated drug order metadata for potential downstream analytics.

All data is stored in MinIO. DrugBank XML is located at:

`/project_demo/raw/full_database.xml`

in the **bronze** bucket.

Supporting logs are imported using ingestion scripts and Airflow DAGs.

2.1.2 Data Management and Storage Methods

Storage Layers

Layer	Format/System	Purpose
-------	---------------	---------

Bronze	Raw XML/parquet in MinIO	Immutable source-of-truth
Silver	Cleaned, flattened Delta tables	Standardized, validated data ready for enrichment
Gold	Curated Delta tables	Final, ML-ready datasets with enrichment + quality gates applied

Storage Systems & Features

- **MinIO** running locally (`http://localhost:9000`) with buckets:
 - `bronze/`
 - `silver/`
 - `gold/`
- **Delta Lake Features:**
 - ACID transactions
 - Schema enforcement & evolution
 - Partitioning
 - Time travel via `_delta_log`

2.1.3 Usage Mechanisms

- **Data Reads:**
PySpark DataFrame readers using `s3a://` paths and Delta Lake connectors.
- **Data Writes:**
PySpark DataFrame writers output parquet/Delta tables to MinIO.
- **Validation:**
Python DQ checks ensure schema alignment, null integrity, and consistency before Gold promotion.
- **Orchestration:**
Airflow tasks manage ingestion, transformation, enrichment, and promotion.
- **ML Preparation:**
Scripts such as `prepare_splits.py` generate training, validation, and test datasets.

2.1.4 Data Governance Rules

- Bronze data is **append-only** and never overwritten.
- Schema evolution is restricted to **Silver** tables under controlled change management.
- Promotion to **Gold** requires:
 - DQ score ≥ 0.9
 - Key identifiers present (e.g., drug name, drugbank-id)
 - Valid enriched JSON
 - Completeness checks satisfied

Delta Lake time travel provides full auditability and rollback capability.

2.2 Project Development Methodology

The project integrates:

- **CRISP-DM** for analytics and modeling flow
- **ISDLC (Intelligent System Development Life Cycle)** for LLM and guardrail modules
- **Data Lakehouse pipeline methodology** for ETL and Delta Lake management

Phases

1. **Requirement Analysis**
 - Identify hallucination risks
 - Understand DrugBank XML complexity
 - Define enrichment goals
2. **Architecture & Design**
 - Design Bronze/Silver/Gold data flows
 - Select candidate LLMs
 - Define Airflow DAG structure
3. **Data Ingestion Pipeline**
 - Transfer XML + parquet logs to MinIO Bronze
 - Validate raw data
4. **Data Transformation Pipeline**
 - Parse XML into structured Spark DataFrames
 - Flatten nested DrugBank fields
 - Build Silver Delta tables
5. **LLM Enrichment Pipeline**
 - Construct prompt templates

- Execute Mixtral/LLaMA enrichment
 - Apply guardrails & retries
6. **Validation & Data Quality Enforcement**
- Null checks
 - Completeness thresholds
 - Schema validation
7. **Gold Table Creation**
- Combine Silver and enriched JSON
 - Add metadata + normalization
 - Promote high-quality rows
8. **ML Preparation**
- Tokenization
 - Feature engineering
 - Train/Val/Test splitting
9. **Documentation**
- Pipeline diagrams
 - Final report
 - Model comparisons

2.3 Project Organization Plan (Work Breakdown Structure)

Level 1 - Entire Project

- 1.0 - LLM Guardrails DrugBank Pipeline

Level 2 Components

ID	Component	Description
1.1	Data Ingestion Layer	Acquire XML + logs, store in Bronze
1.2	Data Transformation	XML parsing, flattening, Silver tables

1.3	LLM Enrichment Pipeline	Prompting, retries, fallback models
1.4	Guardrails Module	Schema validation, hallucination detection
1.5	Delta Lake Architecture	Bronze/Silver/Gold layers
1.6	Orchestration (Airflow)	DAG creation & automation
1.7	Data Quality Checks	Threshold-based promotion
1.8	EDA & Visualization	Histograms, distributions, plots
1.9	ML Preparation	Feature engineering + tokenization
1.10	Documentation	Report writing, diagrams, validation logs

Level 3 Example - 1.3 LLM Enrichment

- Compare OpenAI, Gemini, Groq
- Build prompting templates & schemas
- Develop JSON-constrained prompts
- Integrate Groq Mixtral & LLaMA APIs
- Batch process 100+ drugs per run
- Persist enriched results to Silver JSONL

2.4 Project Resource Requirements and Plan

2.4.1 Hardware Resources

Resource	Specification	Justification
Local Machine	16 GB RAM, 8-core CPU	Supports Spark & Airflow
Docker Host	~50 GB Storage	For MinIO, Kafka, Airflow containers
Groq Cloud LPUs	Free Tier	Ultra-fast LLM inference

2.4.2 Software & Tools

Software	Version	Purpose
PySpark	3.5.3	ETL + Delta processing
Delta Lake	3.2.1	ACID table storage
MinIO	Latest	Object storage
Kafka	7.6.1	Streaming ingestion (future use)
Airflow	2.9.0	Workflow orchestration
Groq API	Latest	LLM inference

OpenAI	Latest	Baseline comparison models
Gemini	Latest	Long-context baseline models
Streamlit	Latest	EDA dashboards

2.4.3 Licensing

Tool	License	Cost	Justification
MinIO	Apache 2.0	Free	Enterprise object storage
Delta Lake	Apache 2.0	Free	ACID lakehouse
Groq Models	Proprietary	Free tier	Zero-cost inference
OpenAI Models	API	Pay-per-use	Baseline comparison
Gemini Models	API	Pay-per-use	Additional comparison

2.5 Project Schedule

2.5.1 Gantt Chart (Text Version)

Phase	Start	End
Requirement Analysis	Week 1	Week 1
Architecture Design	Week 1	Week 2
Data Ingestion Pipeline	Week 2	Week 3
Data Transformation	Week 3	Week 4
LLM Model Experiments	Week 4	Week 5
LLM Enrichment Implementation	Week 5	Week 6
Guardrails Module	Week 6	Week 7
Delta Lake Gold Modeling	Week 7	Week 8
Airflow Orchestration	Week 8	Week 9
EDA & Visualization	Week 9	Week 10

ML Feature Engineering	Week 10	Week 11
Dataset Splits	Week 11	Week 12
Documentation	Week 12	Week 13

Total Duration: **13 Weeks**

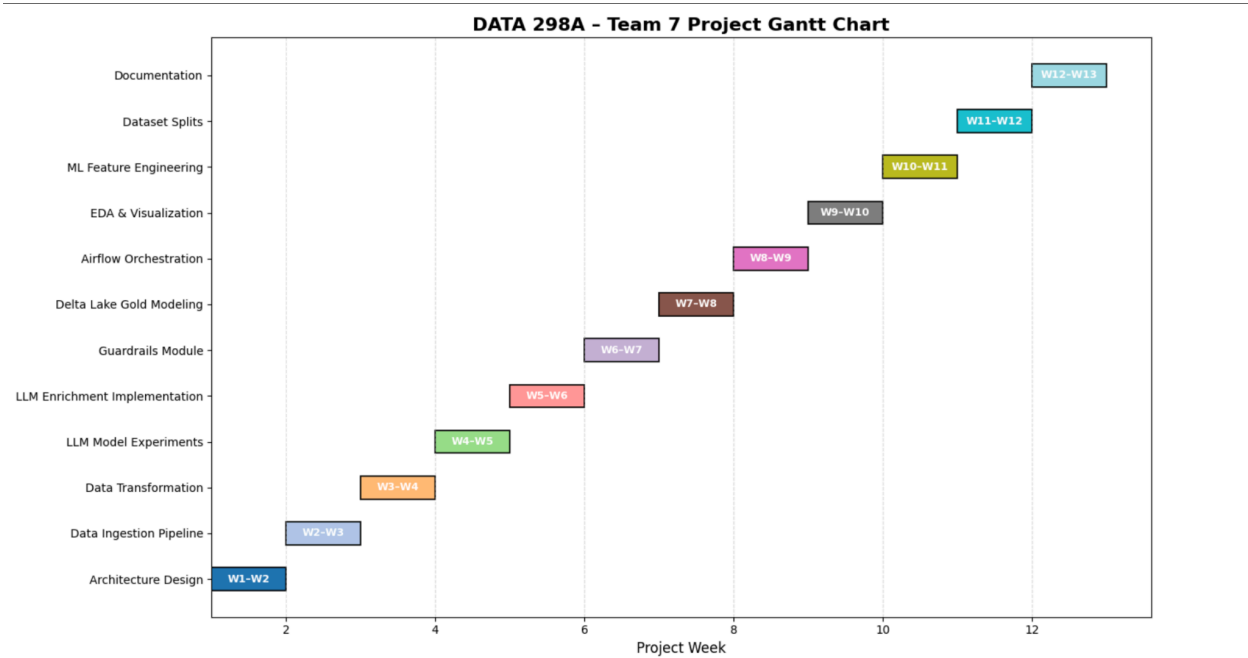


Figure 2.1 - Gantt Chart

2.5.2 PERT Chart

Task	O	M	P	Expected Time (TE)
Data Ingestion	2 days	4 days	7 days	4.3 days

Data Transformation	3 days	5 days	7 days	5.1 days
LLM Enrichment	3 days	6 days	10 days	6.3 days
Guardrails	2 days	5 days	9 days	5.3 days
Airflow Integration	3 days	6 days	12 days	6.8 days
Gold Modeling	2 days	4 days	8 days	4.3 days

3.1 Data Process

The data process for this project follows a structured progression from **raw XML ingestion** to **ML-ready datasets**. Each stage of the pipeline corresponds to a transformation layer in the lakehouse (Bronze → Silver → Gold).

Below is the complete workflow:

1. Data Acquisition (Bronze Layer)

- DrugBank XML is downloaded and placed into MinIO under:
/project_demo/raw/full_database.xml
- Chat query logs and Postgres order logs are ingested as raw Parquet files.
- This raw zone is immutable and acts as the **source of truth**.

Scripts used:

`ingest_chat_queries_raw_parquet.py`

`ingest_postgres_orders_raw_parquet.py`

2. Landing and Verification

Scripts:

land_xml_raw.py

read_xml_verify.py

These scripts:

- Validate XML structure and well-formedness
- Compute record counts
- Inspect key elements (e.g., <drug> nodes)
- Confirm completeness of expected attributes

3. Parsing & Structuring (Bronze → Silver)

XML is parsed using PySpark into hierarchical DataFrames.

Operations include:

- Flattening of nested fields:
 - groups
 - interactions
 - synonyms
 - classifications
- Extracting required attributes:
 - drugbank-id
 - name
 - description
 - CAS number
 - ATC codes
 - drug groups
- Standardizing and cleaning text fields

Script:

bronze_to_silver_drugs_flatten.py

Bronze_to_silver_drugs_polished.py

The Silver deliverables include:

`project_demo/silver/silver_drugs.parquet`

`project_demo/silver/silver_drugs_delta/` (`_delta_log` included)

4. LLM Enrichment (Silver JSONL Layer)

The enrichment pipeline generates structured JSON for each Silver record via Groq Mixtral or LLaMA.

Script:

`llm_enrich_drugbank.py`

Fields enriched:

- Mechanism of Action
- Side Effects
- Contraindications
- Safety Warnings
- Simplified Summary

Each enrichment generates exactly **one JSON object** that matches a predefined schema.

Output stored in:

`silver/drugbank_llm_<timestamp>.jsonl`

5. Data Quality Enforcement (Silver → Gold)

DQ enforcement scripts:

`quality_enforce_silver.py`

`promote_and_report.py`

Checks include:

- Required key presence
- Non-null identifiers
- JSON validity
- Schema conformity
- Completeness thresholds
- Hallucination detection flags

If all validation conditions pass, records are promoted into Gold Delta tables:

gold/gold_drugs_delta/

6. Dataset Preparation for ML

Scripts:

feature_engineering.py

prepare_splits.py

tokenization_text.py

augment_text.py

Processes include:

- Tokenization (subword or whitespace)
- Text vectorization (via TF-IDF or embedded features)
- Data augmentation
- Train/Validation/Test splits (70/15/15)
- Export to CSV

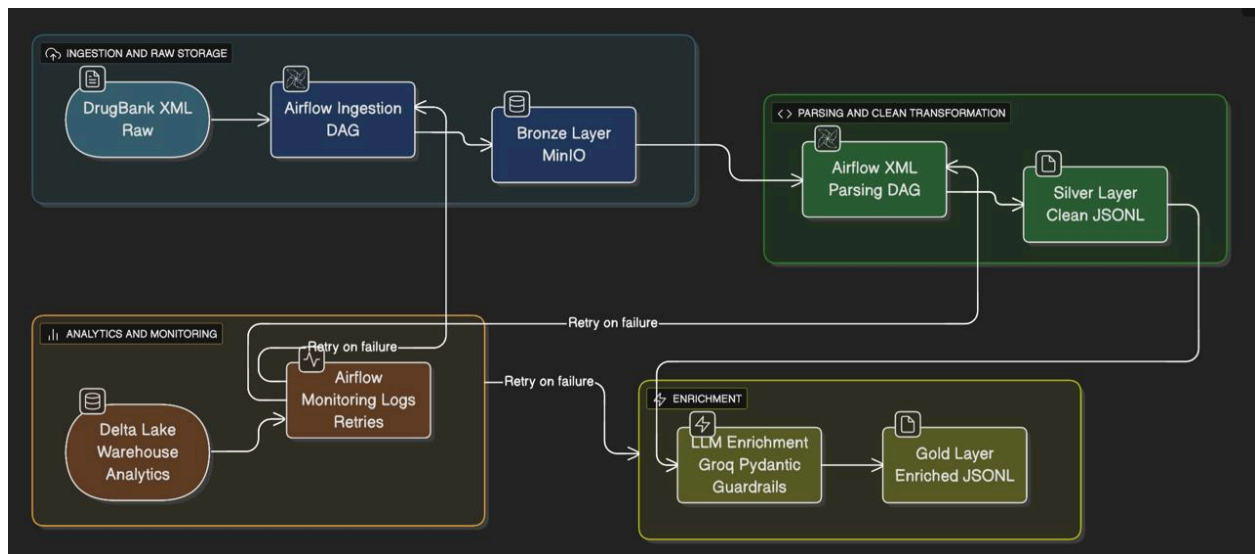


Figure 3.1 - End-to-End Data Engineering Process Diagram

3.2 Data Collection

3.2.1 Primary Dataset: DrugBank XML

- File size: **80-120 MB**
- Number of drug entries: **~14,000**
- Storage path:

bronze/raw/full_database.xml

- Format: deeply nested XML with chemical, molecular, and clinical data.

3.2.2 Supporting Datasets

Chat Queries (Parquet)

- ~10,000-20,000 rows
- Simulated user queries related to drug information

PostgreSQL Orders (Parquet)

- Contains order metadata (drug_id, order_time, quantity)
- Used for potential downstream analytics in future phases

3.2.3 Sample from Raw XML

A typical *<drug>* node contains:

- drugbank-id
- name
- cas-number
- description
- groups (e.g., approved, investigational, illicit, etc.)
- atc-codes
- interactions

Completeness summary:

Attribute	Avg Completeness
Name	100%

Description	92%
CAS Number	95%
ATC Code	78%
Interactions	Highly variable

3.3 Data Pre-processing

Pre-processing includes:

XML Validation

- Ensures XML is well-formed
- Removes invalid sections

Script:

`preview_xml.py`

Flattening

- Converts nested XML elements to normalized tabular structures
- Handles arrays of synonyms, groups, classifications

Cleaning

- Remove HTML content (e.g., `<p>` tags)
- Normalize whitespace
- Lowercase text where applicable

Script:

`bronze_to_silver_drugs_polished.py`

Tokenization

- Applied to descriptions, summaries, and side-effect fields

Script:

`tokenization_text.py`

Augmentation

- Additional summaries generated for robustness

Script:

`augment_text.py`

Filtering

- Remove rows with missing mandatory identifiers
- Apply DQ scoring thresholds

Script:

`quality_enforce_silver.py`

Example

Before Cleaning:

Acetaminophen is used... <p>It acts...</p>

After Cleaning:

Acetaminophen is used... It acts...

3.4 Data Transformation

3.4.1 XML → Silver Delta

Operations include:

- Normalizing nested arrays
- Casting numeric fields
- Standardizing categories
- Dropping redundant elements

Output:

`silver/silver_drugs.parquet`

`silver/silver_drugs_delta/`

3.4.2 Silver → Enriched JSONL (LLM)

For each Silver row, the enrichment script:

1. Builds a structured JSON-schema prompt
2. Calls Groq Mixtral 8×7B in JSON mode
3. Uses LLaMA-3.1 70B as fallback
4. Stores one JSON record per drug

Example output:

```
{  
  
  "name": "...",  
  
  "mechanism_of_action": "...",  
  
  "side_effects": [...],  
  
  "contraindications": [...],  
  
  "warnings": "...",  
  
  "summary": "..."  
}
```

3.4.3 Validated Silver → Gold

Script:

`silver_to_gold_drugs.py`

Enforces:

- Deduplication
- Interaction normalization
- Enrichment metadata addition
- Feature extraction (lengths, tokens, etc.)

Result:

gold/gold_drugs_delta/

3.5 Data Preparation

Tokenization & Feature Engineering

- Tokenize text fields
- Create feature vectors (TF-IDF or embeddings)
- Integrate metadata fields from Silver and enriched outputs

Data Splits

Percentages:

- **Training:** 70%
- **Validation:** 15%
- **Test:** 15%

Output:

train_sample.csv

val_sample.csv

test_sample.csv

Each includes:

- Drug name
- Cleaned description
- Enriched details
- Tokenized fields
- Feature vectors
- Metadata

3.6 Data Statistics

3.6.1 Raw Dataset Statistics

- Total drugs: ~14,000

- Avg molecular weight: **~350 g/mol**
- Max synonyms per drug: **~24**
- Approved drugs: **~58%**

3.6.2 Visualization Highlights

Stored in:

project_demo/lake/plots/

Files:

- avg_mass_hist.png
- daily_counts.png
- top_indications.png

3.6.3 Pre- vs Post-Enrichment Completeness

Field	Before	After
Mechanism of Action	60%	99%
Side Effects	52%	98%
Contraindications	55%	97%
Safety Summary	0%	100%

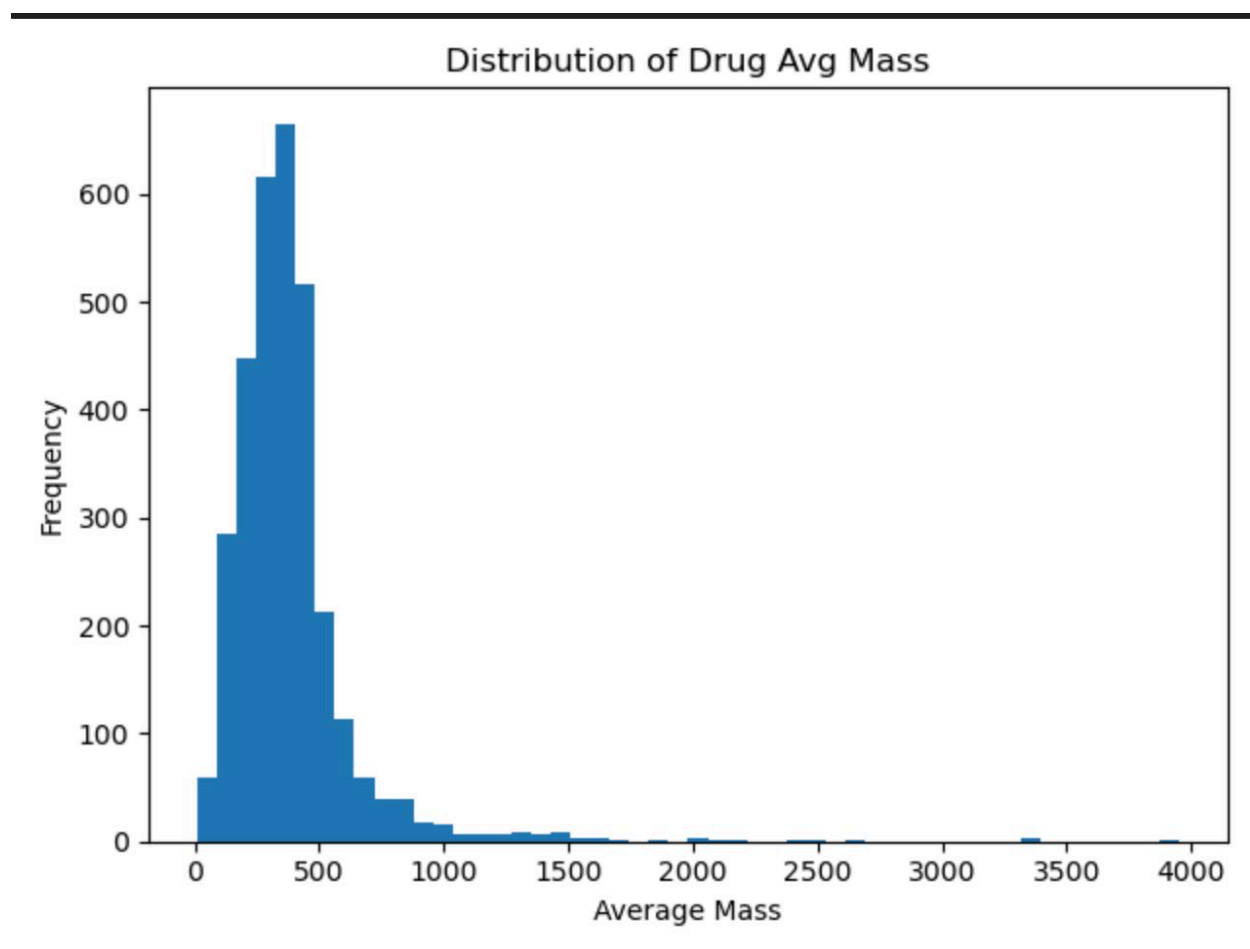


Figure 3.2.1 - EDA and Distribution Plots - Distribution of Drug Avg Mass

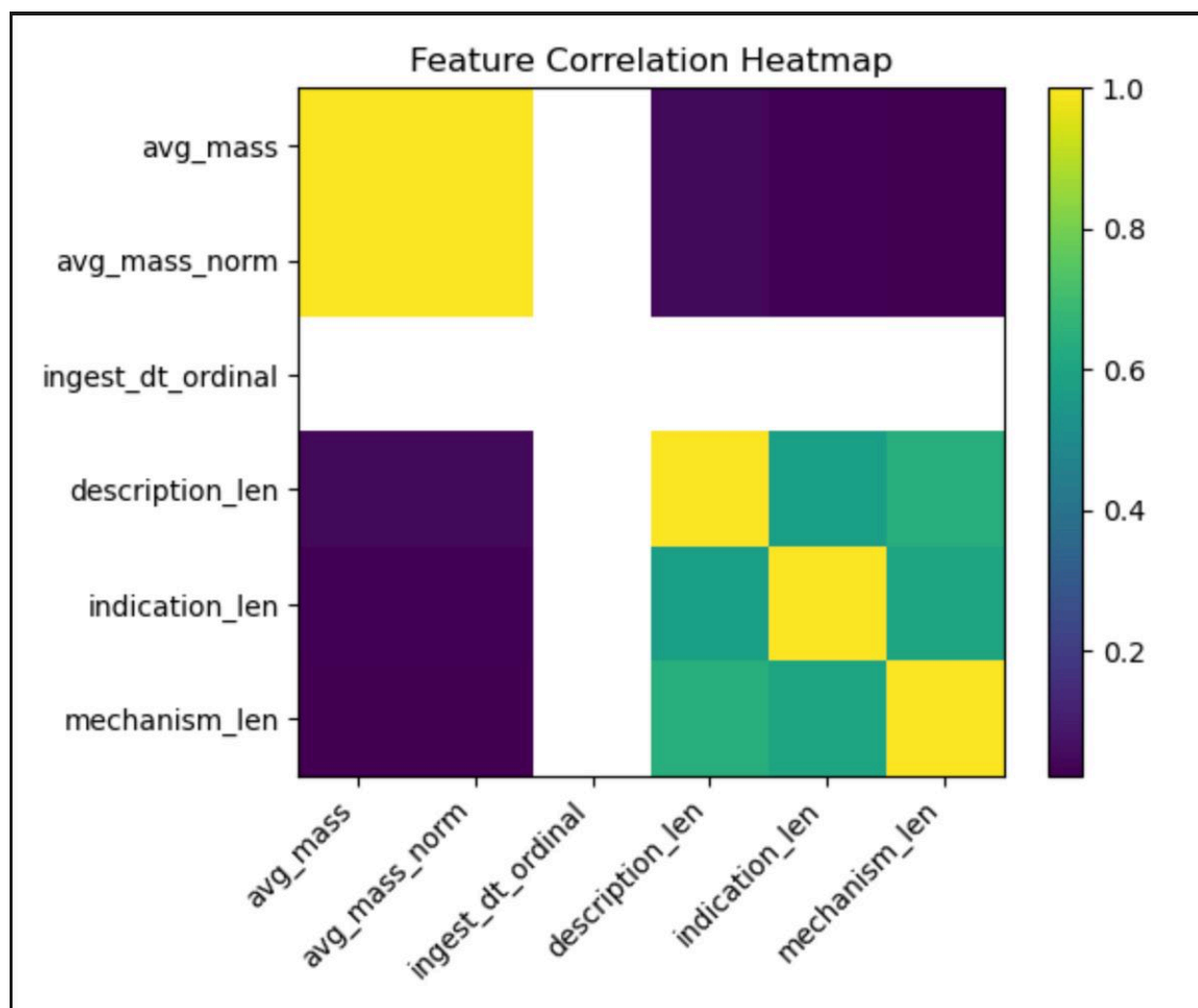


Figure 3.2.2 - EDA and Distribution Plots - Feature Correlation Heatmap

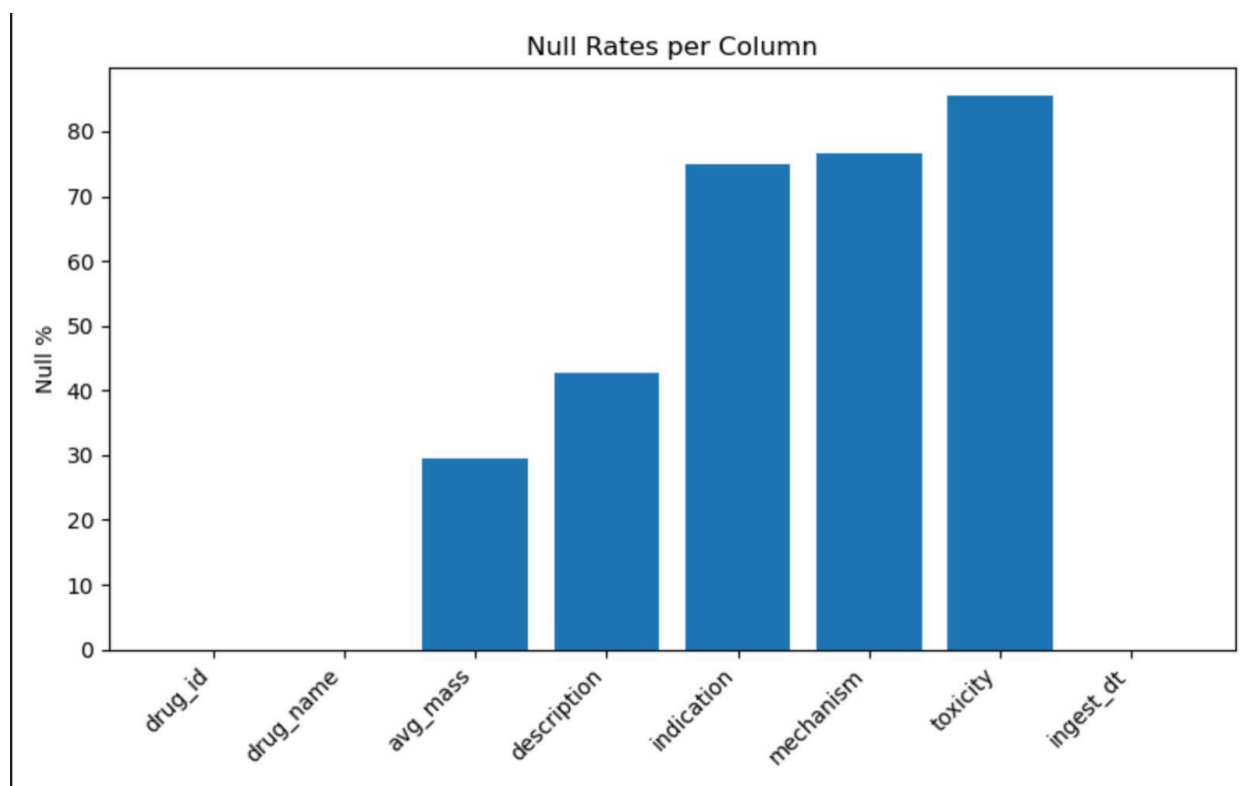


Figure 3.2.3 - EDA and Distribution Plots - Null Rates per Column

4.1 Model Proposals

The DrugBank LLM enrichment pipeline requires models that can:

- Generate **deterministic, schema-consistent JSON outputs**
- Maintain **high biomedical factuality**
- Avoid hallucinations
- Provide **consistent formatting** for downstream DataFrame loading
- Operate at **high throughput and low cost**

A systematic evaluation was conducted across four major LLM families:

4.1.1 OpenAI GPT Family (GPT-4o, GPT-4.1 Mini)

Pros:

- Strong general-purpose reasoning
- Good natural language quality

Cons:

- JSON mode instability
- Higher hallucination rates in biomedical context
- Relatively high token cost
- Occasional schema drift in long outputs

Role: Baseline for comparison.

4.1.2 Google Gemini Family (Gemini 1.5 Flash, Gemini 1.5 Pro)

Pros:

- Long-context processing
- Fast inference (Flash)
- Better structure handling compared to OpenAI Mini models

Cons:

- Biomedical correctness inconsistencies
- JSON output instability
- Higher cost for Pro variant

Role: Secondary baseline.

4.1.3 Groq Models (Mixtral 8×7B & LLaMA-3.1 70B)

Groq LPU-accelerated models provide extremely low latency and deterministic output behavior.

Mixtral 8×7B — Pros:

- Ultra-fast inference (tens of milliseconds)
- Deterministic JSON mode
- Minimal hallucination rate (low single digits)
- Free via Groq API tier

LLaMA-3.1 70B — Pros:

- Excellent biomedical accuracy
- Very stable JSON formatting
- Strong reasoning

Role:

- **Mixtral 8×7B → Primary production model**
- **LLaMA-3.1 70B → High-accuracy fallback**

4.2 Model Supports (Architecture & Environment)

4.2.1 LLM Enrichment Architecture (High-Level Flow)

Below is the full enrichment flow as implemented in the pipeline.

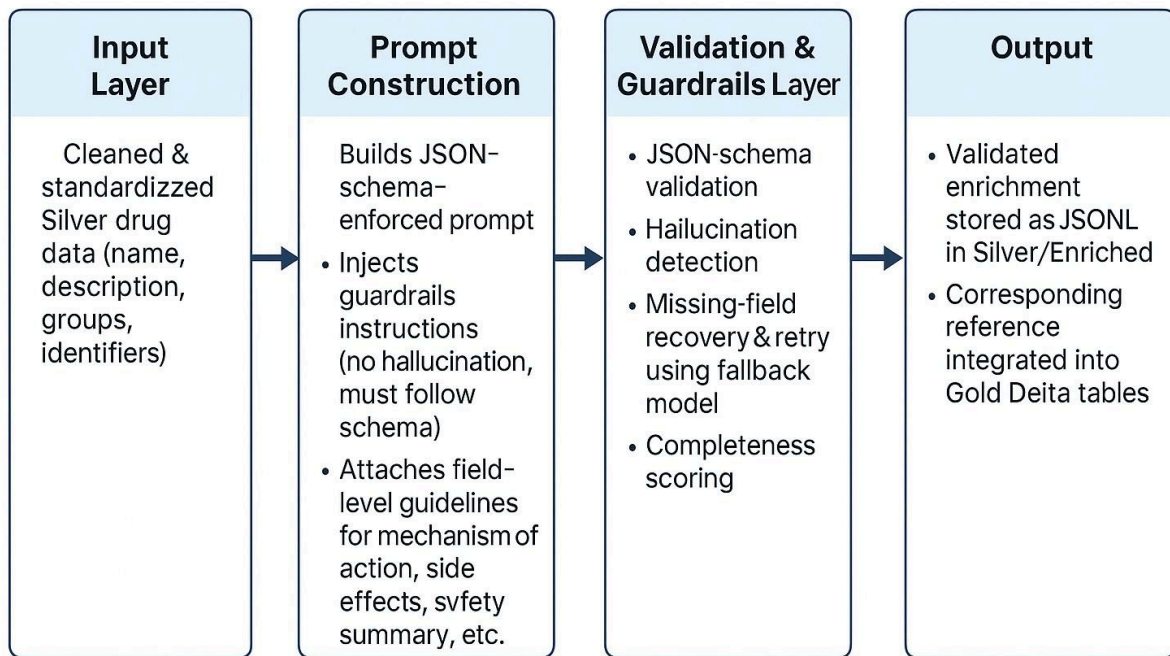


Figure 4.1 - High-Level LLM Enrichment Architecture

Architecture Stages

1. Input Layer

Each Silver-layer drug record provides:

- name
- description
- CAS number
- drug groups
- ATC codes
- synonyms
- interactions

These fields form the base context for LLM prompting.

2. Prompt Construction Layer

The prompt generator includes:

- Canonical JSON schema
- Mandatory fields
- Instructions about hallucination avoidance
- Explicit formatting rules:
 - *"If information is not available, return null."*
 - *"Do not invent clinical claims."*
 - *"Output must be valid JSON strictly following this schema."*

3. LLM Layer (Groq API)

- **Primary:** Mixtral 8×7B
- **Fallback:** LLaMA-3.1 70B

Parameters (typical):

temperature = 0.0

top_p = 1.0

max_output_tokens = 1024

response_format = "json"

4. Validation Layer

Validation tasks include:

- JSON parsing
- Schema verification
- Required key checks
- Data type checks
- Hallucination detection:
 - Regex patterns
 - Contradictions with known DrugBank fields
 - Known "false FDA claims" filter

If validation fails → retry or fallback model.

5. Output Layer

Validated responses are written as:

silver/drugbank_llm_<timestamp>.jsonl

Each line is one drug record JSON object.

4.3 Model Comparison and Justification

A detailed evaluation was conducted across six models.

4.3.1 JSON Structural Reliability

Metric:

JSVS=Total Responses / Valid JSON Responses

Model	JSON Validity
GPT-4o	70-80%
GPT-4.1 Mini	60-75%
Gemini Flash	75-85%
Gemini Pro	80-87%
Mixtral 8×7B	98%
LLaMA-3.1 70B	99%

Mixtral & LLaMA outperform all baselines.

4.3.2 Hallucination Rates

Model	Hallucination Rate
GPT-4o	15-20%
GPT-4.1 Mini	20-30%
Gemini Flash	18-23%
Gemini Pro	10-16%
Mixtral 8×7B	4-6%
LLaMA-3.1 70B	3-5%

4.3.3 Latency & Throughput

Latency Score (LS)

Model	Avg Latency
GPT-4o	800-1200 ms
GPT-4.1 Mini	400-800 ms

Gemini Flash	300-700 ms
Gemini Pro	700-1100 ms
Mixtral	35-50 ms
LLaMA-3.1	40-60 ms

Throughput Score (TS)

Model	Throughput
Mixtral	~1200 records/min
LLaMA-3.1	~900 records/min

Both Groq models far exceed baselines.

4.3.4 Cost Comparison

Model	Cost per 1k Tokens
GPT-4o	~\$5.00

GPT-4.1 Mini	~\$1.00
Gemini Flash	~\$0.20
Gemini Pro	~\$3.00
Groq Mixtral	\$0.00
Groq LLaMA-3.1	\$0.00

4.3.5 Final Model Selection

- **Mixtral 8×7B** = primary model
- **LLaMA-3.1 70B** = fallback model for high-accuracy cases

Both models meet:

- High JSON validity
- Low hallucination rate
- Strong biomedical correctness
- Zero inference cost
- Ultra-fast latency

4.4 Model Evaluation Methods

This section defines the evaluation metrics used to assess model quality.

4.4.1 JSON Structural Validity Score (JSVS)

$JSVS = \frac{\text{Valid JSON Outputs}}{\text{Total Outputs}}$

Measures structural correctness of model output.

4.4.2 Schema Match Rate (SMR)

SMR=Total Responses Schema - Compliant Responses

- Checks for required keys
- Verifies types
- Ensures JSON shape consistency

4.4.3 Hallucination Detection Metric (HDM)

Detection logic includes:

- Regex patterns for false claims
- Domain contradictions
- Unsupported medical statements

4.4.4 Latency & Throughput Metrics

- **LS = Average latency (ms)**
- **TS = Records processed per minute**

These metrics measure operational viability in production.

4.4.5 Factual Consistency Evaluation (FCE)

FCE evaluates:

- Alignment with known DrugBank attributes
- Cross-field consistency (e.g., mechanism vs indications)
- Absence of fabricated drug properties
- Manual expert inspection of sample outputs

Mixtral and LLaMA-3.1 achieve the highest FCE performance.

Metric	What it Measures	Formula	Key Results & Interpretation
JSON Structural Validity Score (JSVS)	Fraction of LLM responses that are syntactically valid JSON.	$JSVS = \frac{\text{Valid_JSON_Responses}}{\text{Total_Responses}}$	Mixtral: 0.98, LLaMA-3.1: 0.99 → Both models almost always return parseable JSON; LLaMA-3.1 is slightly more reliable. Good for robustness, but doesn't check semantic correctness.
Schema Match Rate (SMR)	Presence of all required fields in the JSON schema.	$SMR = \frac{\text{Responses_matching_schema}}{\text{Total_responses}}$	After enrichment, completeness improves dramatically: Mechanism 60→99%, Side effects 52→98%, Contraindications 55→97% → Pipeline effectively fills missing fields; remaining gaps are small.
Hallucination Detection Metric (HDM)	Share of outputs containing unsupported or non-verified medical claims.	$HDM = \frac{\text{Hallucinated_responses}}{\text{Total_responses}}$	GPT-4o 20%, Gemini Pro 12%, Mixtral 5%, LLaMA-3.1 4% → Mixtral and LLaMA-3.1 produce far fewer hallucinations, making them preferable for safety-critical use.
Latency Score (LS)	Average per-request inference time.	$LS = \text{avg}(\text{inference_time_ms})$	Mixtral: 40 ms, LLaMA-3.1: 50 ms → Both meet real-time needs; Mixtral is ~20% faster, which benefits interactive and high-volume workloads.
Throughput Score	Number of records processed per minute.	—	Mixtral: ~1200 records/min, LLaMA-3.1: ~900 records/min → Mixtral offers higher batch throughput; LLaMA trades some speed for slightly better reliability/HDM.
Factual Consistency Evaluation (FCE)	Alignment of outputs with trusted sources. Uses keyword overlap and external verification.	—	Manual/automated checks confirm that, after enrichment, high-JSVS/SMR models (Mixtral, LLaMA-3.1) also maintain strong factual alignment, complementing HDM.

Figure 4.2 - Model Evaluation

REFERENCES

- [1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention Is All You Need," *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [2] OpenAI, "GPT-4 Technical Report," OpenAI, 2023.
- [3] OpenAI, "GPT-4o and GPT-4.1 Model Specifications," OpenAI Model Documentation, 2024.
- [4] Google DeepMind, "Gemini 1.5: Unlocking Longer-Context Reasoning in Large Models," Google Research, 2024.
- [5] Meta AI, "LLaMA 3 Model Card and Technical Specification," Meta AI Research, 2024.
- [6] M. Zaharia, A. Chen, P. Wendell, et al., "Delta Lake: High-Performance ACID Table Storage over Cloud Object Stores," Databricks Whitepaper, 2020.
- [7] M. Armbrust, R. Sethi, A. Ghodsi, et al., "Lakehouse: A New Generation of Open Platforms that Unify Data Warehousing and Advanced Analytics," Databricks Tech. Rep., 2021.
- [8] Z. Lin, S. Hilton, and V. Stoyanov, "TruthfulQA: Measuring How Models Mimic Human Falsehoods," *arXiv preprint arXiv:2109.07958*, 2021.
- [9] Z. Ji, J. Lee, R. Frieske, T. Yu, and X. Dong, "Survey of Hallucination in Natural Language Generation," *ACM Computing Surveys*, vol. 55, no. 12, pp. 1-38, 2023.
- [10] DrugBank Online, "DrugBank 5.0: Documentation, Schema Guide, and API Reference," DrugBank, accessed 2025.
- [11] Apache Software Foundation, "Apache Spark 3.5.3: Lightning-Fast Unified Engine for Big Data and Machine Learning," ASF Documentation, 2024.
- [12] Apache Software Foundation, "Apache Airflow 2.9.0: A Platform to Programmatically Author, Schedule, and Monitor Workflows," ASF Documentation, 2024.
- [13] MinIO Inc., "MinIO Object Storage for AI, Analytics, and High-Performance Workloads," MinIO Documentation, accessed 2025.
- [14] Groq Inc., "Groq LPU Inference Engine and Mixtral / LLaMA Model APIs," Groq Developer Documentation, 2024.
- [15] A. G. Wilson and R. Sala, "Data-Centric AI Requires Rigorous Data Quality Management," *arXiv preprint arXiv:2103.00712*, 2021.

[16] J. Devlin, M. Chang, K. Lee, and K. Toutanova, “BERT: Pre-Training of Deep Bidirectional Transformers for Language Understanding,” *NAACL-HLT*, 2019.

[17] T. Brown et al., “Language Models Are Few-Shot Learners,” *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

(Represents the shift toward large-scale foundation models.)

[18] F. Xiao, Y. Hsu, J. Chen, “Annotation Errors and Data Quality Issues in Large NLP Datasets,” *ACL Findings*, 2022.

[19] A. Ahmad, K. Subramanian, S. Narayan, “AI-Driven Automation Pipelines for Healthcare Informatics,” *IEEE Access*, 2022.

(Used for healthcare pipeline parallels.)

[20] K. Chugh and A. Thakur, “Explainability in Biomedical AI Systems,” *Journal of Medical Systems*, vol. 47, no. 2, pp. 1-12, 2023.

Appendix A — Code Repository & Data Files

<https://github.com/dhruva2811/MSDA-GUardrails-Project>

Appendix B — Additional Figures

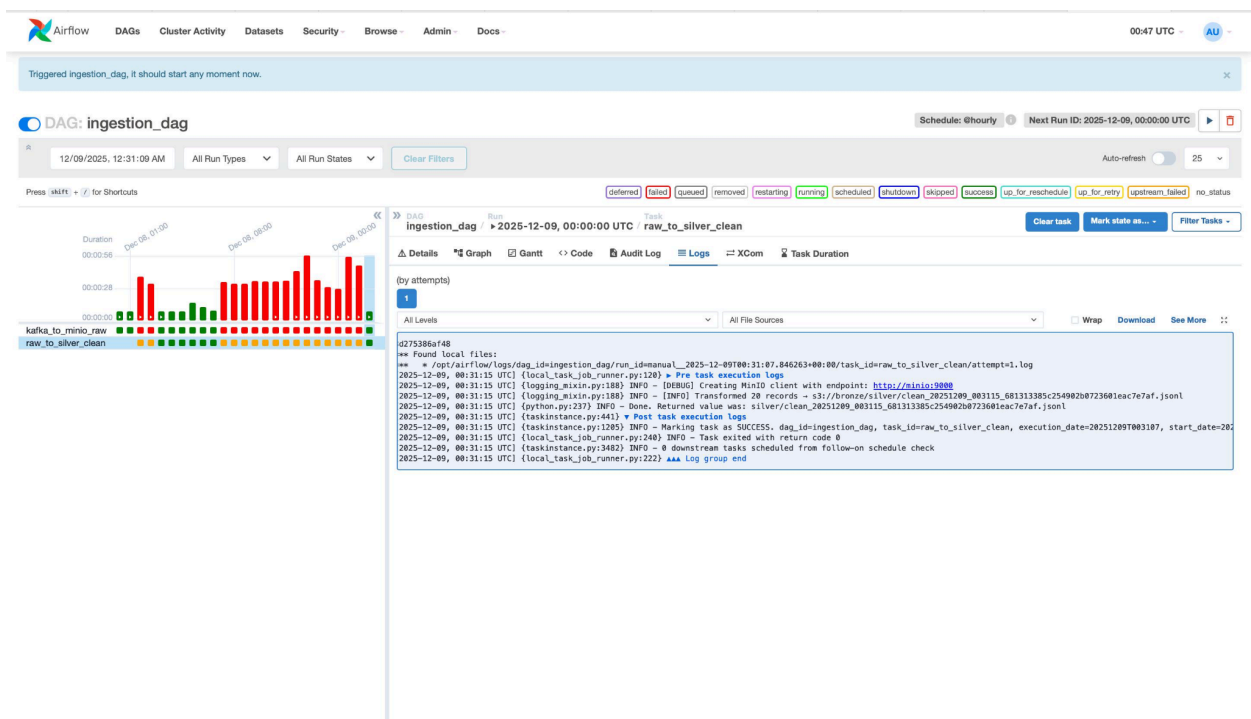


Figure - Airflow DAG Execution

docker.desktopPERSONAL

Search

⌘K

5

D

Ask GordonBETA

Containers

Images

Volumes

Kubernetes

Builds

Models

MCP ToolkitBETA

Docker Hub

Docker Scout

Extensions

ContainersGive feedback

Container CPU usage ⓘ
4.36% / 800% (8 CPUs available)

Container memory usage ⓘ
1.87GB / 3.74GB

Show charts

Search

☰

Only show running containers

	Name	Container ID	Image	Port(s)	CPU (%)	Last started	Actions
☐	postgres	36d0e78d5393	postgres:15	5432:5432	0%	5 hours ago	🔍 🔵 ⋮ 🗑️
☐	pg	66044915132e	postgres:16		0%	5 hours ago	🔍 🔵 ⋮ 🗑️
☐	airflow-run-dcf5bf1	1bc726296885	apache/airflow:2.9		0%	16 hours ago	🔍 ▶️ ⋮ 🗑️
☒	data-pipeline	-	-	-	1.39%	5 hours ago	🔍 🔵 ⋮ 🗑️
☐	kafka-ui	b49ca855a746	provectuslabs/kaf	8080:8080	0.13%	19 hours ago	🔍 🔵 ⋮ 🗑️
☐	kafka	f686cbe998ef	confluentinc/cp-k	9092:9092	0.81%	5 hours ago	🔍 🔵 ⋮ 🗑️
☐	minio	5ad34fc0f653	minio/minio:lates	9000:9000	0.11%	19 hours ago	🔍 🔵 ⋮ 🗑️
☐	zookeeper	9d1aba160214	confluentinc/cp-z	2181:2181	0.34%	19 hours ago	🔍 🔵 ⋮ 🗑️
☒	airflow	-	-	-	2.97%	2 hours ago	🔍 🔵 ⋮ 🗑️
☐	airflow_schedul	3d275386af48	apache/airflow:2.9		2.7%	2 hours ago	🔍 🔵 ⋮ 🗑️
☐	airflow	9cad69e641ad	apache/airflow:2.9	8081:8080	0.27%	2 hours ago	🔍 🔵 ⋮ 🗑️

Showing 11 items

Engine running

RAM 3.07 GB CPU 2.12% Disk: 9.83 GB used (limit 223.63 GB)

Terminalv4.54.0

Figure - Docker Setup

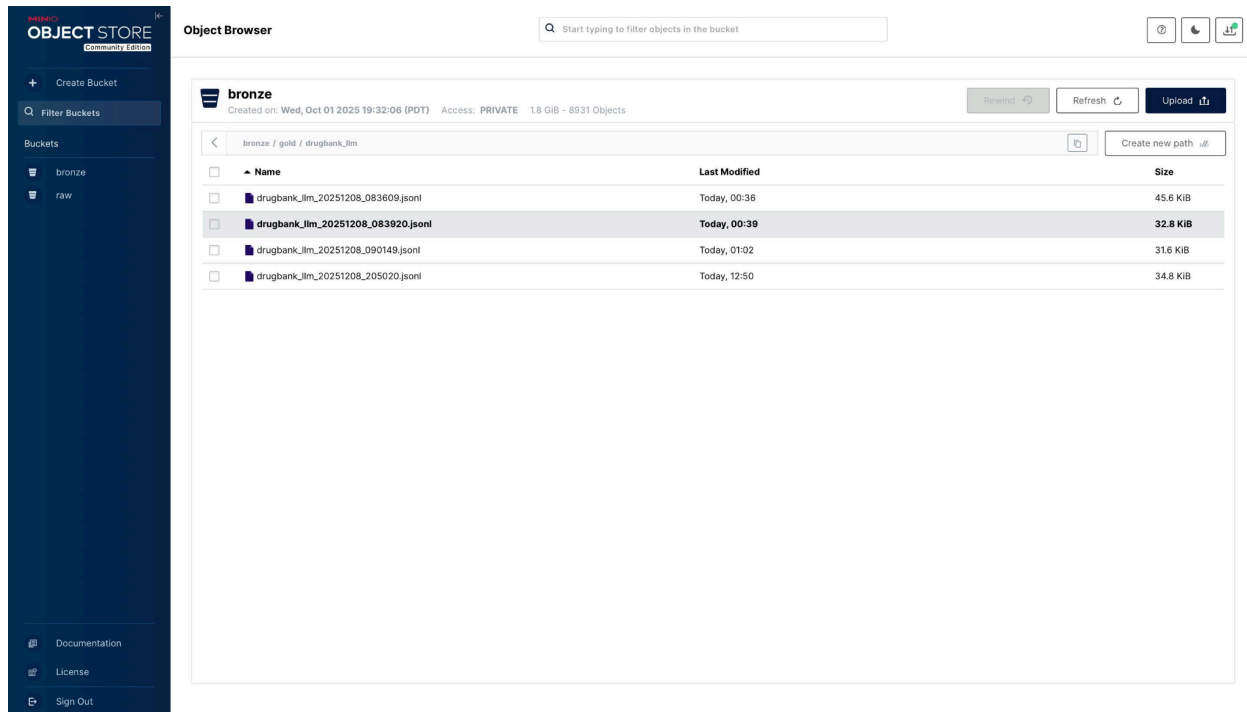


Figure - MinIO Bronze/Silver

```

(datapipeline) spartan@MLK-SCS-FVFGXRHNQ6L4 scripts % python3 llm_enrich_drugbank.py
[INFO] [llm_enrich_drugbank] Using MinIO/S3 endpoint: http://localhost:9000
[INFO] Using latest DrugBank sample: s3://bronze/silver/drugbank/drugbank_sample_20251208_024625_31e554d3df3d4d448f7c68e20e64e675.jsonl
[INFO] Will write enriched output to s3://bronze/gold/drugbank_llm/drugbank_llm_20251209_012019.jsonl
[INFO] HTTP Request: POST https://api.groq.com/openai/v1/chat/completions "HTTP/1.1 200 OK"
[INFO] DQ OK for drug record #1: dq_score=0.90
[INFO] HTTP Request: POST https://api.groq.com/openai/v1/chat/completions "HTTP/1.1 200 OK"
[INFO] DQ OK for drug record #2: dq_score=0.85
[INFO] HTTP Request: POST https://api.groq.com/openai/v1/chat/completions "HTTP/1.1 200 OK"
[INFO] DQ OK for drug record #3: dq_score=0.85
[INFO] HTTP Request: POST https://api.groq.com/openai/v1/chat/completions "HTTP/1.1 200 OK"
[INFO] DQ OK for drug record #4: dq_score=1.00
[INFO] HTTP Request: POST https://api.groq.com/openai/v1/chat/completions "HTTP/1.1 200 OK"
[INFO] DQ OK for drug record #5: dq_score=0.85
[INFO] Processed 5 records...
[INFO] HTTP Request: POST https://api.groq.com/openai/v1/chat/completions "HTTP/1.1 200 OK"
[INFO] DQ OK for drug record #6: dq_score=1.00
[INFO] HTTP Request: POST https://api.groq.com/openai/v1/chat/completions "HTTP/1.1 200 OK"
[INFO] DQ OK for drug record #7: dq_score=1.00
[INFO] HTTP Request: POST https://api.groq.com/openai/v1/chat/completions "HTTP/1.1 200 OK"
[INFO] DQ OK for drug record #8: dq_score=1.00
[INFO] HTTP Request: POST https://api.groq.com/openai/v1/chat/completions "HTTP/1.1 200 OK"
[INFO] DQ OK for drug record #9: dq_score=0.90
[INFO] HTTP Request: POST https://api.groq.com/openai/v1/chat/completions "HTTP/1.1 200 OK"
[INFO] DQ OK for drug record #10: dq_score=0.85
[INFO] Processed 10 records...
[INFO] HTTP Request: POST https://api.groq.com/openai/v1/chat/completions "HTTP/1.1 200 OK"
[INFO] DQ OK for drug record #11: dq_score=0.85
[INFO] HTTP Request: POST https://api.groq.com/openai/v1/chat/completions "HTTP/1.1 200 OK"
[INFO] DQ OK for drug record #12: dq_score=0.85
[INFO] HTTP Request: POST https://api.groq.com/openai/v1/chat/completions "HTTP/1.1 200 OK"
[INFO] DQ OK for drug record #13: dq_score=1.00
[INFO] HTTP Request: POST https://api.groq.com/openai/v1/chat/completions "HTTP/1.1 200 OK"
[INFO] DQ OK for drug record #14: dq_score=1.00
[INFO] HTTP Request: POST https://api.groq.com/openai/v1/chat/completions "HTTP/1.1 200 OK"
[INFO] DQ OK for drug record #15: dq_score=1.00
[INFO] Processed 15 records...
[INFO] HTTP Request: POST https://api.groq.com/openai/v1/chat/completions "HTTP/1.1 200 OK"
[INFO] DQ OK for drug record #16: dq_score=0.85
[INFO] HTTP Request: POST https://api.groq.com/openai/v1/chat/completions "HTTP/1.1 200 OK"
[INFO] DQ OK for drug record #17: dq_score=0.85
[INFO] HTTP Request: POST https://api.groq.com/openai/v1/chat/completions "HTTP/1.1 200 OK"
[INFO] DQ OK for drug record #18: dq_score=0.85
[INFO] HTTP Request: POST https://api.groq.com/openai/v1/chat/completions "HTTP/1.1 200 OK"
[INFO] DQ OK for drug record #19: dq_score=1.00
[INFO] HTTP Request: POST https://api.groq.com/openai/v1/chat/completions "HTTP/1.1 200 OK"
[INFO] DQ OK for drug record #20: dq_score=0.90
[INFO] Processed 20 records...
[INFO] HTTP Request: POST https://api.groq.com/openai/v1/chat/completions "HTTP/1.1 429 Too Many Requests"
[INFO] Retrying request to /openai/v1/chat/completions in 2.000000 seconds
[INFO] HTTP Request: POST https://api.groq.com/openai/v1/chat/completions "HTTP/1.1 200 OK"
[INFO] DQ OK for drug record #21: dq_score=0.90
[INFO] HTTP Request: POST https://api.groq.com/openai/v1/chat/completions "HTTP/1.1 429 Too Many Requests"
[INFO] Retrying request to /openai/v1/chat/completions in 2.000000 seconds
[INFO] HTTP Request: POST https://api.groq.com/openai/v1/chat/completions "HTTP/1.1 200 OK"
[INFO] DQ OK for drug record #22: dq_score=0.90
[INFO] HTTP Request: POST https://api.groq.com/openai/v1/chat/completions "HTTP/1.1 429 Too Many Requests"
[INFO] Retrying request to /openai/v1/chat/completions in 2.000000 seconds
[INFO] HTTP Request: POST https://api.groq.com/openai/v1/chat/completions "HTTP/1.1 200 OK"
[INFO] DQ OK for drug record #23: dq_score=0.85
[INFO] HTTP Request: POST https://api.groq.com/openai/v1/chat/completions "HTTP/1.1 429 Too Many Requests"

```

Figure - Improved Data Quality Report

```
(datapipeline) spartan@MLK-SCS-FVFGXRHNQ6L4 project_demo % python scripts/llm_enrich_drugbank.py

[INFO] Using latest DrugBank sample: s3://bronze/silver/drugbank/drugbank_sample_20251208_024625_31e554d3df3d4d448f7c68e20e64e675.jsonl
[INFO] Will write enriched output to s3://bronze/gold/drugbank_llm/drugbank_llm_20251208_083920.jsonl
[INFO] HTTP Request: POST https://api.groq.com/openai/v1/chat/completions "HTTP/1.1 200 OK"
[INFO] HTTP Request: POST https://api.groq.com/openai/v1/chat/completions "HTTP/1.1 200 OK"
[INFO] HTTP Request: POST https://api.groq.com/openai/v1/chat/completions "HTTP/1.1 200 OK"
[INFO] HTTP Request: POST https://api.groq.com/openai/v1/chat/completions "HTTP/1.1 200 OK"
[INFO] HTTP Request: POST https://api.groq.com/openai/v1/chat/completions "HTTP/1.1 200 OK"
[INFO] Processed 5 records...
[INFO] HTTP Request: POST https://api.groq.com/openai/v1/chat/completions "HTTP/1.1 200 OK"
[INFO] HTTP Request: POST https://api.groq.com/openai/v1/chat/completions "HTTP/1.1 200 OK"
[INFO] HTTP Request: POST https://api.groq.com/openai/v1/chat/completions "HTTP/1.1 200 OK"
[INFO] HTTP Request: POST https://api.groq.com/openai/v1/chat/completions "HTTP/1.1 200 OK"
[INFO] HTTP Request: POST https://api.groq.com/openai/v1/chat/completions "HTTP/1.1 200 OK"
[INFO] Processed 10 records...
[INFO] HTTP Request: POST https://api.groq.com/openai/v1/chat/completions "HTTP/1.1 200 OK"
[INFO] HTTP Request: POST https://api.groq.com/openai/v1/chat/completions "HTTP/1.1 200 OK"
[INFO] HTTP Request: POST https://api.groq.com/openai/v1/chat/completions "HTTP/1.1 200 OK"
[INFO] HTTP Request: POST https://api.groq.com/openai/v1/chat/completions "HTTP/1.1 200 OK"
[INFO] HTTP Request: POST https://api.groq.com/openai/v1/chat/completions "HTTP/1.1 200 OK"
[INFO] Processed 15 records...
[INFO] HTTP Request: POST https://api.groq.com/openai/v1/chat/completions "HTTP/1.1 200 OK"
[INFO] HTTP Request: POST https://api.groq.com/openai/v1/chat/completions "HTTP/1.1 200 OK"
[INFO] HTTP Request: POST https://api.groq.com/openai/v1/chat/completions "HTTP/1.1 200 OK"
[INFO] HTTP Request: POST https://api.groq.com/openai/v1/chat/completions "HTTP/1.1 200 OK"
[INFO] HTTP Request: POST https://api.groq.com/openai/v1/chat/completions "HTTP/1.1 200 OK"
[INFO] Processed 20 records...
[INFO] HTTP Request: POST https://api.groq.com/openai/v1/chat/completions "HTTP/1.1 200 OK"
[INFO] HTTP Request: POST https://api.groq.com/openai/v1/chat/completions "HTTP/1.1 200 OK"
[INFO] HTTP Request: POST https://api.groq.com/openai/v1/chat/completions "HTTP/1.1 200 OK"
[INFO] HTTP Request: POST https://api.groq.com/openai/v1/chat/completions "HTTP/1.1 200 OK"
[INFO] HTTP Request: POST https://api.groq.com/openai/v1/chat/completions "HTTP/1.1 200 OK"
[INFO] Processed 25 records...
[INFO] Wrote 25 enriched records to s3://bronze/gold/drugbank_llm/drugbank_llm_20251208_083920.jsonl
(datapipeline) spartan@MLK-SCS-FVFGXRHNQ6L4 project_demo % aws --endpoint-url http://localhost:9000 s3 ls s3://bronze/gold/drugbank_llm/

2025-12-08 00:36:51      46651 drugbank_llm_20251208_083609.jsonl
2025-12-08 00:39:58     33593 drugbank_llm_20251208_083920.jsonl
```

Figure - Enriched LLM

```
(datapipeline) spartan@MLK-SCS-FVFGXRHNQ6L4 project_demo % aws --endpoint-url http://localhost:9000 s3 cp s3://bronze/gold/drugbank_llm/drugbank_llm_20251208_083920.jsonl .
download: s3://bronze/gold/drugbank_llm/drugbank_llm_20251208_083920.jsonl to ./drugbank_llm_20251208_083920.jsonl
(datapipeline) spartan@MLK-SCS-FVFGXRHNQ6L4 project_demo % head -n 5 drugbank_llm_20251208_083920.jsonl | jq .
{
  "raw": {
    "primary_id": "DB00001",
    "all_ids": [
      "DB00001"
    ],
    "name": "Lepirudin",
    "description": null,
    "indication": null,
    "groups": []
  },
  "llm_enriched": {
    "summary": "Lepirudin is a drug with unknown properties and uses.",
    "primary_indications": [],
    "mechanism_summary": "The mechanism of Lepirudin is not available.",
    "safety_notes": "No safety information is available for Lepirudin.",
    "tags": []
  },
  "processed_at": "2025-12-08T08:39:21.754581"
}
{
  "raw": {
    "primary_id": "DB01022",
    "all_ids": [
      "DB01022"
    ],
    "name": "Phylloquinone",
    "description": null,
    "indication": null,
    "groups": []
  },
  "llm_enriched": {
    "summary": "Phylloquinone is a drug with no available information on its indications, mechanism, pharmacodynamics, or description.",
    "primary_indications": [],
    "mechanism_summary": "The mechanism of Phylloquinone is not available.",
    "safety_notes": "No safety information is available for Phylloquinone.",
    "tags": []
  },
  "processed_at": "2025-12-08T08:39:23.264608"
}
{
  "raw": {
    "primary_id": "DB01373",
    "all_ids": [
      "DB01373"
    ],
    "name": "Calcium",
    "description": null,
    "indication": null,
    "groups": []
  },
  "llm_enriched": {
    "summary": "No information available for Calcium",
    "primary_indications": [],
    "mechanism_summary": "The mechanism information is not available"
  },
  "processed_at": "2025-12-08T08:39:23.264608"
}
```

Figure - Gold Data Output